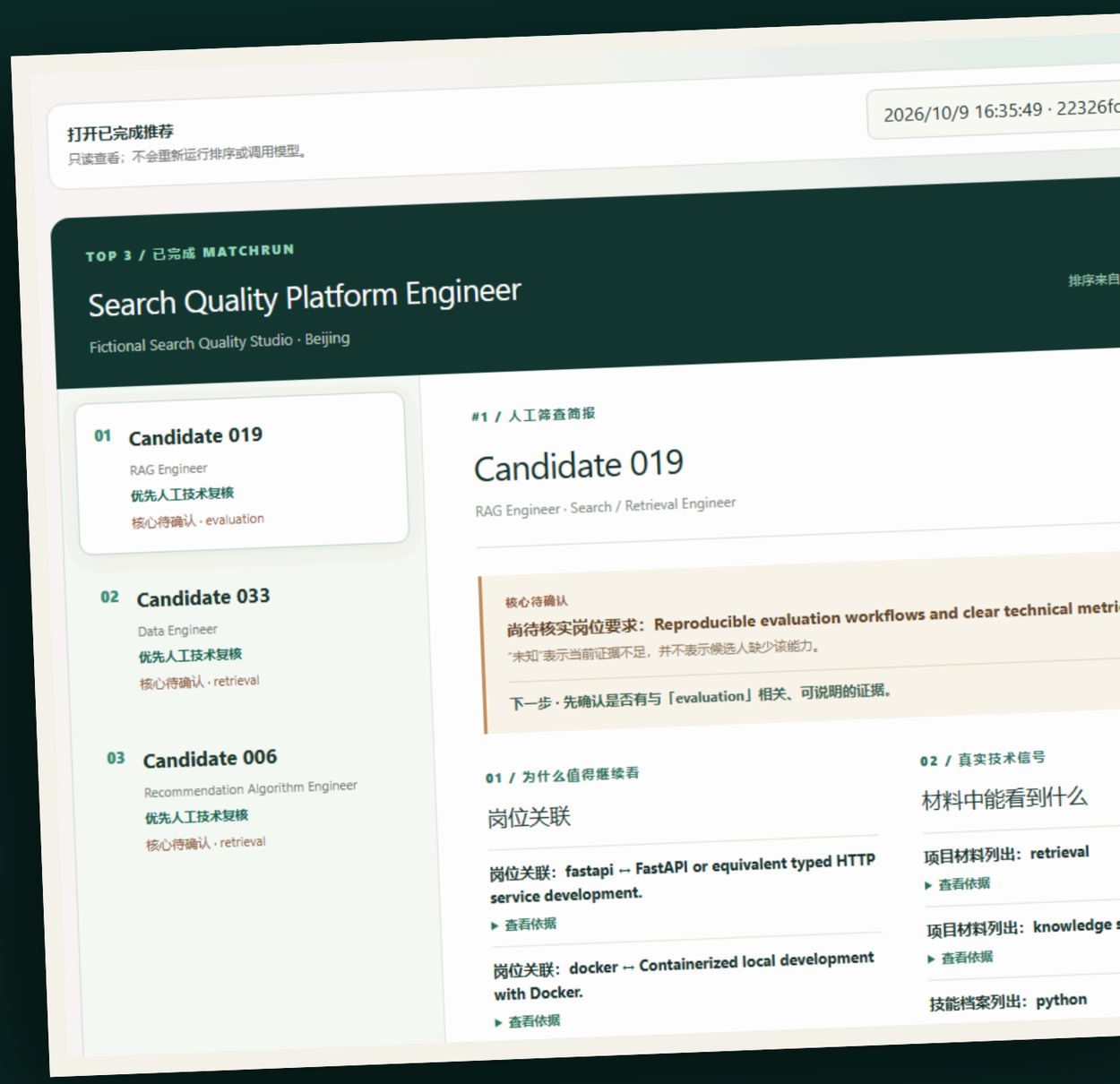
AI ALGORITHM × LLM SYSTEMS × AGENTIC ENGINEERING

我设计 AI 产品。也设计它被可靠交付的方式。

PeopleMatch 连接检索、排序、可追溯解释与人工核实；我把新功能交给 Codex 实现，再用审查、测试和真实页面决定是否交付。

PEOPLEMATCH AGENT

AI Builder 人岗匹配与证据复核系统



HISTORICAL BASE + V5 SOURCE IN THIS REVISION · LOCAL SYNTHETIC PROTOTYPE

THE DECISION SURFACE

不是一个分数。
是一张能复核的候选名单。

排序只是起点。Recruiter View 同时给出证据、核心未知与下一步核实目标。



01 / ACTION

优先人工技术复核

02 / SIGNAL

材料中可见的技术证据

03 / UNKNOWN

Evaluation 能力待核实

04 / NEXT CHECK

先问可说明的项目证据

BOUNDARY / 不是录用建议，也不是匹配或 Offer 概率。

01

/ MISSION

FROM A REAL RECRUITER LIMITATION

不能只告诉招聘者“先核实”。要告诉他核实哪一项。

原 Recruiter View 只突出少数 Unknown。一次筛查需要逐项看到岗位 must-have / nice-to-have、现有材料与未建立的关系。

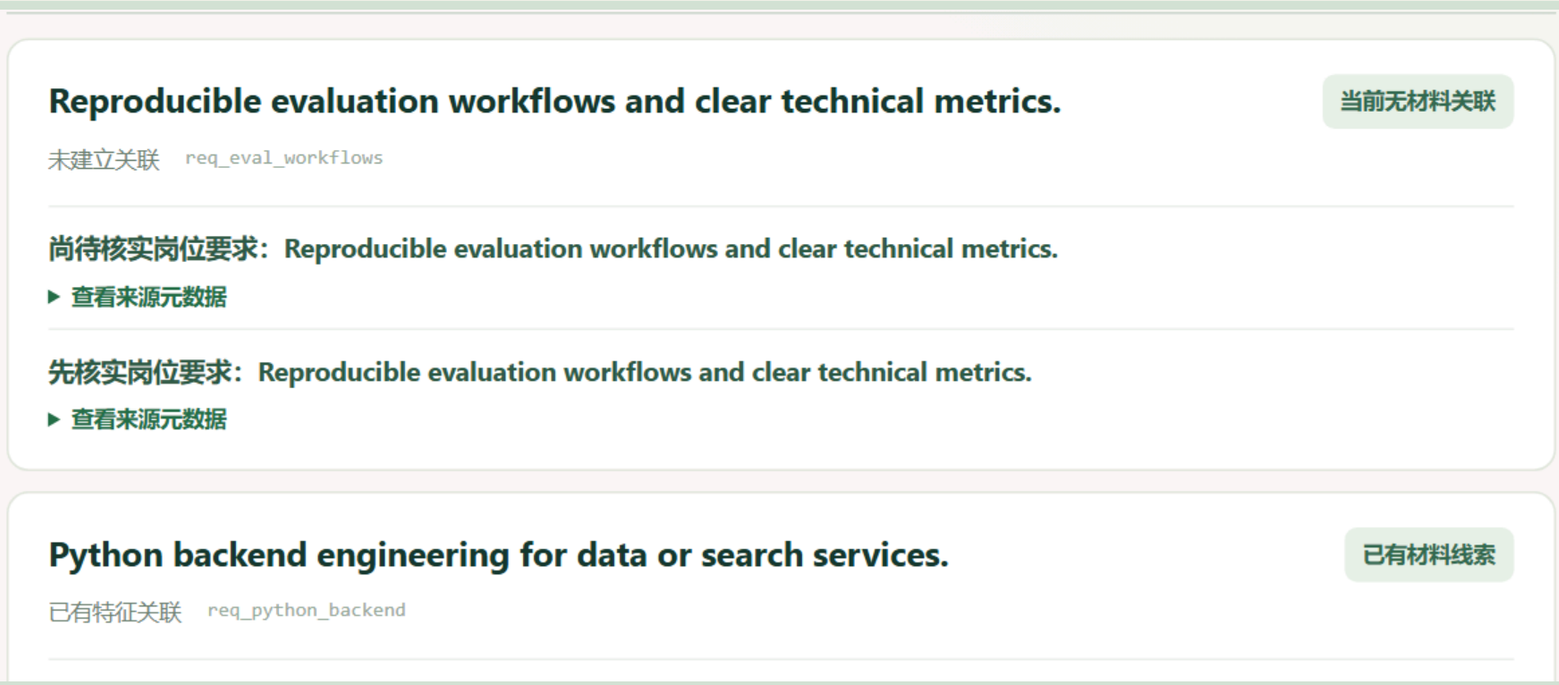
INPUT	NEW PRODUCT WORK	NON-GOAL
已保存的 JD + Evidence Bundle	**逐项来源核实工作台**	**不重排、不补造能力事实**

我选的是招聘判断链上的缺口，不是为了展示 Codex 而加一个技术面板。

N / V5 NEW IMPLEMENTATION

REAL POSTGRESQL READBACK · REAL NEXT.JS PAGE

一条岗位要求，
材料、未知与核实动作同屏。



01 / ACTUAL BROWSER · MUST-HAVE + UNKNOWN

SCREEN DETAIL / EDITED TEXT

有来源，
才谈判断。

诊断状态与精选解释是两条不同线索。Unknown 表示材料关联尚未建立，不等于候选人缺少能力。

人工复核路径

1. 查看历史来源元数据
2. 确认材料与岗位要求的关联
3. 对 Unknown 提出定向核实问题

02 / HUMAN CHECK · EDITED TRANSCRIPT

03 / SAVED SOURCE · EDITED TRANSCRIPT

SAVED SOURCE / EDITED TEXT **历史档案技能声明：python** 岗位原文：Python backend engineering for data or search services.

真实浏览器主图；右侧和下方为同页内容的编辑转录 · 历史 synthetic Fresh Run 的 PostgreSQL 只读回读；V5 源码见本版本。

ONE RUN · THREE DISTINCT OWNERS

召回、排序、解释：不是同一个决定。

Qwen 参与抽取与解释选择；正式 Top-3 由检索、显式特征与 LambdaMART 决定。

INPUT / FACTS

Candidate + JD

Structured Extraction

→

RANKING OWNER

Hybrid Retrieval

Dense + Sparse / RRF

10 Pair Features

LambdaMART → Top-3

→

IMMUTABLE SEAM

MatchRun

order · raw score · fingerprint

→

EXPLANATION OWNER

Evidence Bundle

Scoped Semantic Selection

Deterministic Binder

Plan → Grounding → Proposition

LLM-OWNED

语义抽取与有限选项选择

CODE-OWNED

身份、排名、引用、关系、来源

REVIEWER-OWNED

招聘判断与事实核实

RETRIEVAL FINDS · FEATURES EXPLAIN · LTR ORDERS

相似度只是候选池入口，不是最终顺序。



ONE VISIBLE RUN

#3 → #1

retrieval position → final rank

RANKING MODEL RAW SCORE

0.180453

LTR margin · NOT a match probability

FORMAL FEATURE CONTRACT / 10

semantic · sparse · skill fit · project evidence · role direction · AI tools · location · seniority · missing penalty · overclaim penalty

截图只显示代表性数值，不是完整特征表。

EXPLANATION MUST HAVE A WAY BACK

一句推荐理由，必须能找到出处。

模型可选择有来源的语义重点；程序绑定并呈现，不能把未展示的能力写成已验证事实。



READ THE PROVENANCE

01 Recruiter Claim

02 Candidate Source

03 Project / Skill Evidence

04 JD Requirement

UNKNOWN
证据未建立 ≠ 没有能力。

DEV DECISION / TEST OBSERVATION

Dev 预先确定默认策略，Test 不重新选型。

R2 Dev 预先选定 LambdaMART。下图只展示冻结 synthetic Test 的事后观察，不能据此重新选型。

TEST R2 / JUDGED-CONDENSED NDCG@5 · 数值越高越好



PREDECLARED DEV LEADER / R2

LTR 保持默认。CE 留作实验。

冻结报告将 Dev leader 与 Test observed ordering 分列。Test 结果并未重新选出 LTR；CE 也并非普遍无效。

A · FROZEN REPORT

LTR NDCG@10	LTR MRR@10	PAIRWISE ACCURACY	TEST SCOPE
0.9597	1.0000	1.0000	5 jobs / 25%

Synthetic sandbox · Post-exposure Test · 288 explicit judgments across the frozen 24×48 base · Unlabeled ≠ negative · Not production hiring accuracy

PHASE 9 / RUNTIME QWEN · NOT A CODEX CODING FAILURE

错误不只在 JSON。更在职责分配。

完整对象方案看起来省事：让 Qwen 一次写完解释。但程序早已知道身份、排名和证据关系，却要求模型再输出一遍。

BEFORE / ONE MODEL OUTPUT HAD TWO JOBS

真正需要语言判断	**哪些理由、风险、核实目标值得说?**
程序已经知道	**candidate • rank • refs • summary / comparison 关系**

A / DOCUMENTED PROVIDER CONTRACT FAILURES

`Provider` invalid_output

历史截图审计记有一次被拒绝的 Provider 运行；工程决策记录概述了引用与跨字段契约失败。原始失败 body 未保留，不重画具体字段错误。

ENGINEERING DIAGNOSIS

继续加 Prompt 或事后 Regex，仍让模型拥有这些关系。

更可靠的方案是收回这部分决定权：让模型做语义选择，让代码绑定确定性结构，Grounding 保留最后的来源校验。

THE DECISION / GIVE EACH COMPONENT THE RIGHT AUTHORITY

我不再让模型重写程序已经知道的关系。

缩小 Qwen 的输出契约，不是放松检查：选择由模型完成；身份、引用与关系由程序绑定；Grounding 继续追问来源。

BEFORE / FULL OBJECT

模型兼做选择与结构组装。

- candidate identity / rank
- selected reasons / refs
- summary / comparison
- cross-field invariants

契约失败可能先于 Grounding；Prompt 不能单独保证跨字段一致。

AFTER / SCOPED SEMANTIC SELECTION

RUNTIME QWEN

选 Handle

从候选人专属 Catalog 中选理由、风险与核实目标。

CODE / BINDER

组关系

身份、rank、规范 refs、顺序、summary 与 comparison。

GROUNDING

再核验

来源、候选归属和可引用事实；越界 fail closed。

semantic_selection.py · SELECTED ACTUAL LINES / NON-CONTIGUOUS

```
option = options.get(handle)
if option is None:
    raise SemanticSelectionError("selection_unknown_or_cross_candidate_handle")
"candidate_ref": source.candidate_ref,
"result_rank": source.result_rank,
"evidence_refs": list(primary.value.evidence_refs),
```

Unknown / wrong-type → reject

`Handle` 不能伪造来源。

Identity / rank → source

不从模型正文回填。

Plan → Grounding

解释不能改历史排名。

BUILD-TIME CODEX ≠ RUNTIME QWEN

我把 Coding Agent 的工作，设计成可验收的工程链。

不以工具调用次数衡量能力；从产品问题到实际运行，每一段都有责任与拒绝条件。

问题定义 → 任务契约 → 相关上下文 → 工具执行
反馈诊断 → 独立审查 → 状态交接 → 验收

V5 / ACTUAL

Mission 先于代码

冻结 Ranking、Grounding、历史 MatchRun。

REVIEW / ACTUAL

测试绿了仍查来源

历史数据暴露 HTTP 200 后的客户端拒绝。

ACCEPT

系统行为决定交付

PostgreSQL、GET、可信客户端、浏览器共同证明。

SOURCE 真实文件 **HISTORICAL** 保存的旧验证 **CURRENT** 本轮重跑 **RECONSTRUCTED** 产物复盘 **NEW LAB** 隔离实验

ACTUAL MISSION → ACTUAL FILES → ACCEPTANCE

不是给 Codex 一句需求。是给它一项能被拒绝的任务。

engineering/mission.md — WRITTEN BEFORE IMPLEMENTATION · V5 LOCAL

01 / USER VALUE	**逐项看 must-have、材料线索与 Unknown；指向下一步人工核实。**
02 / READ FIRST	**历史 Evidence Bundle • Requirement diagnostics • Proposition • Recruiter View**
03 / FROZEN	**Ranking / Grounding / MatchRun 不改；不推断缺失技能，不写历史数据。**
04 / ACCEPT	**错候选、错引用、错指纹必须拒绝；API 和浏览器读同一历史来源。**

CODING AGENT EXECUTION SPACE

Service	只读来源投影
GET API	类型化响应与 409
Frontend	Requirement + 来源下钻

最终验收 ≠ 代码生成结束

RED → review → PostgreSQL → browser

上下文只选相关仓库模块；实际任务、命令和修复保存在 V5 Mission / execution / review 文件中，非历史逐字聊天。

REPRESENTATIVE ENGINEERING RECONSTRUCTION

让 Agent 先回答：这个事实属于谁？

任务不是“读完整仓库”，而是沿一条来源链定位所有权；以下指令由真实 Context Map 编辑，不是原始聊天。

REPOSITORY INVESTIGATION / BEFORE EDITING	
01 / JOB	**定位历史 JD 要求与当前 Job Profile：哪一个能给旧 Run 提供原文?**
02 / EVIDENCE	**追踪保存的 Evidence Bundle → Proposition → Explanation identity。**
03 / CLIENT	**阅读 trusted-recruiter 的拒绝条件；列出 API 必须满足的来源闭包。**
OUTPUT	**交付事实所有权表、候选隔离链与最小改动文件；在确认前不写 UI。**

WHY THIS MATTERED

```
Mutable Profile ≠ historical source

Aggregate ≠ one requirement

HTTP 200 ≠ trusted payload
```

后续独立审查与 PostgreSQL 回读，正是在这些边界发现问题。

THE ENGINEERING ENVIRONMENT / PHASE 6

先让 Codex 读懂仓库，再交出实现权。

工作现场由四类真实文件组成：项目规则、阶段计划、冻结特征契约和当前数据状态；工具只在这些边界内执行。

CONTEXT / PHASE 6

从 Phase 5 接过已冻结输入。

先读契约与现有源码，再决定需要检索、编辑和验证的最小文件集合。

rM0on / task context

CONTEXT SELECTED FOR THIS TASK

RULES	AGENTS.md → TDD / 不越界 / 报告真实验证
SPEC	Phase 6 plan → 输入、交付物、禁止修改
CONTRACT	Phase 5 feature order + fingerprint
STATE	Train/Dev labels · Hybrid pool · frozen Test

AVAILABLE CHECKS → REVIEW

READ	仓库检索 · source · plan
RUN	pytest · artifact / metadata checks
REVIEW	Git diff · refusal paths

工具提供反馈；
契约决定是否接受。

CODING AGENT / EXECUTES ---→ CONTRACT + EVIDENCE / DECIDE ACCEPTANCE

A / rules + plan + code · B / editorial workbench reconstruction

PHASE 0–12 / VERSIONED HANDOFF

跨阶段开发，让产物接力，而不是让对话接力。

Phase 5 输出的特征契约，成为 Phase 6 的输入；Phase 6 的模型、拒绝性测试和元数据，再成为 Phase 7 可审查的起点。

00–03	04–05	06–07	08–10	11–12
定义 & 数据	**召回 & 特征**	**排序 & 对照**	**运行 & 解释**	**评测 & 展示**
PRD → contracts → PostgreSQL → extraction	Hybrid pool → 10-feature fingerprint	Train/Dev LTR → optional CE	MatchRun → grounded plan → review	frozen report → product verification

REAL TRANSFER / PHASE 5 → 6

给下一轮任务的不是“继续做”。是一个可接手的状态包。

01 / INPUT

已验证特征契约

MODEL_FEATURE_ORDER
十项顺序 + fingerprint
Phase 5 测试与冻结文档

02 / FREEZE

不能漂移的边界

不改 Phase 4/5、标签或 split；Test 不进训练、调参或方案选择。

03 / EXIT GATE

交付给 Phase 7

dataset / trainer / model，拒绝性测试、native model metadata 和可追溯分值。

恢复任务时核对现有计划、源码、测试和产物，不从旧聊天猜状态。图中交接关系由仓库文件重建，并非历史自动调度日志。

V5 / ACTUAL SAVED STATE

下一轮接手时，先校验状态，再继续执行。

V5 用 MISSION、STATE、DECISIONS、HANDOFF、RESUME 记录任务状态。
恢复时核对文件、Diff 与测试；这不是自动多 Agent 平台。

READ

Mission + State

目标、冻结边界、完成项。

→

COMPARE

Git + Diff

确认代码与报告相符。

→

RUN

Targeted tests

对照失败与最新结论。

DECISIONS / WHY

为何保持只读、为何不绕过客户端来源校验。

RESUME / NEXT

保存下一条命令与未验收项；恢复时不把上一轮摘要当证明。

REPRESENTATIVE ENGINEERING RECONSTRUCTION / BASED ON REAL PHASE 6 PLAN

让 LambdaMART 安全训练，先把 Test 挡在门外。

真实书面计划被编辑为可执行任务示范：目标、允许修改的模块、拒绝条件和退出门禁，都能对应后续源码与测试。

CODING AGENT TASK / PHASE 6 · EDITED SPECIMEN

REAL PLAN + SOURCE-BASED RECONSTRUCTION

OBJECTIVE **在冻结十项特征上训练一个确定性 LambdaMART。**	READ FIRST **Feature Contract • Train/Dev judgments • Hybrid pool。**
OWN **dataset / trainer / loader / focused tests / metadata。**	FORBIDDEN **Test、unjudged=negative、标签/split、Phase 4/5 改动。**
TEST FIRST **错误 split / 顺序 / fingerprint 必须被拒绝。**	DONE WHEN **raw margin、stable tie、native model SHA 与验收报告。**

EXIT CRITERIA / NOT A TRAINING SCREENSHOT

输入不合法，模型不训练。

`matrix_for_split` 拒绝 Test 与顺序漂移；`train_ranker` 核验指纹与重复模型 SHA。

PLAN	真实书面任务
CODE	dataset / trainer / model
TEST	拒绝性断言

TASK → SOURCE → NEGATIVE TEST → ACCEPTANCE

交给 Codex 的实现，要先学会拒绝错误输入。

Phase 6 的验收不是“训练完成”。关键在于 Test、错误特征顺序和漂移数据进不了模型。

ACTUAL SOURCE / backend/app/services/ranking/dataset.py · L57–61

```
validate_dataset_fingerprint(artifact)
if split not in {"train", "dev"}:
    raise DatasetIntegrityError("Only Train and Dev matrices are permitted")
if tuple(artifact.get("model_feature_order", ())) != MODEL_FEATURE_ORDER:
    raise DatasetIntegrityError("LTR feature order differs from frozen Phase 5")
```

模型只能接收 Train/Dev 与冻结的特征顺序。

ACTUAL TEST / backend/tests/test_ltr_dataset.py · L88–94

```
payload = _payload()
payload["model_feature_order"].reverse()
artifact = attach_dataset_fingerprint(payload)
with pytest.raises(DatasetIntegrityError):
    matrix_for_split(artifact, "train")
with pytest.raises(DatasetIntegrityError):
    matrix_for_split(attach_dataset_fingerprint(_payload()), "test")
```

顺序漂移与 Test split 都不能悄悄进训练。

MODEL CONTRACT

feature fingerprint

trainer.py 拒绝与冻结 Phase 5 不一致的输入。

REPRODUCIBILITY

repeat native SHA

同环境双次训练，模型 SHA 和预测需一致。

ACCEPT / ONLY AFTER

读 Diff + 测拒绝路径

再核对版本化模型元数据与 raw score。

GREEN IS NOT THE ONLY CONDITION

模型训练通过后，我还要审查“绿”从何而来。

Phase 6 的真实测试与冻结契约，构成对 Coding Agent 产物的审查清单；下列 Review 指令为基于产物的复盘。

REPRESENTATIVE REVIEW REQUEST / PHASE 6

```
Inspect the diff before accepting GREEN.
Was the Test split used for fitting or selection?
Did MODEL_FEATURE_ORDER or the fingerprint change?
Were rejection tests skipped or assertions weakened?
Can a reversed feature order still pass the suite?
```

ACTUAL ACCEPTANCE BOUNDARY

```
Test matrix              → REJECT
Wrong feature order      → REJECT
Fingerprint mismatch     → REJECT
```

若答案依赖修改冻结输入、删反例或让 Test 参与选型，任务不能验收。实际防护见 `ranking/dataset.py` 与 `test_ltr_dataset.py`。

AN EXECUTABLE ASSIGNMENT

先写拒绝条件，再让 Agent 实现功能。

MASTER MISSION / REQUIREMENT REVIEW 2026.10.13 · REAL TASK

OBJECTIVE	**对每个候选人逐项显示岗位要求、材料关联与核实路径。**
OWNERSHIP	**新增只读投影、GET API、typed frontend；复用历史来源与 Proposition。**
FORBIDDEN	**不改 Ranking / Grounding；不凭空推断缺失技能；不写历史 MatchRun。**
TEST FIRST	**全量要求、缺来源、错候选、未知要求、重复 ID、二次读取变更。**
DONE WHEN	**API + 浏览器可操作，正反例、回归、Diff 与来源边界全部验收。**

ACTUAL IMPLEMENTATION RED

4 FAIL

新服务与依赖尚不存在；不是把历史系统编成“有 Bug”。

```
pytest test_requirement_review_workbench.py -q
```

随后实现并复验；原始失败保存在执行记录。

AN INDEPENDENT REVIEW CHANGED THE IMPLEMENTATION

代码能运行，来源仍可能被说错。

审查发现三条捷径：历史引用借用当前档案命名、错误 JD ID 被拼成“合法来源”、跨要求命题被铺到单项要求。修复不是隐藏数据，而是重新绑定历史来源。

BEFORE / OBSERVED RISKS

历史 Explanation

＋ 当前可变档案标签

错误 JD ref

＋ 聚合命题逐项展开

看似完整的证据，会指向错误事实。

AFTER / ACTUAL SERVICE GUARDS · THREE NON-CONTIGUOUS EXCERPTS

```
if evidence_ref not in catalog:
    raise ValueError("historical_candidate_evidence_not_found")

if len(proposition.job_requirement_refs) > 1:
    cross_requirement.append(proposition)

if tuple(diagnostic.evidence_refs) != (ref,):
    raise ValueError("requirement_ref_mismatch")
```

来源从保存时的 Bundle 读取；错误引用拒绝，聚合命题单列。

REVIEW → RED → FIX → GREEN **可变档案污染：拒绝** **错误引用：409** **聚合命题：单列**

真实拒绝性测试 / test_requirement_review_workbench.py

ACTUAL SOURCE / THE SOURCE-CLOSURE FIX

能引用，不等于该展示。
我让 API 只交付被使用的来源。

backend/app/services/requirement_review/service.py

EXACT CONTIGUOUS EXCERPT

```
for evidence_ref in proposition.evidence_refs:
    if evidence_ref.startswith("candidate:"):
        if evidence_ref not in catalog:
            raise ValueError("historical_candidate_evidence_not_found")
        if proposition.job_requirement_refs:
            used_sources.add(evidence_ref)
```

验证全部引用　但只把当前命题实际使用的候选来源写进响应。

DELIVERED

Service • GET API • typed UI

INDEPENDENT REVIEW

可变档案 / 错 JD ref / 聚合误归因

HISTORICAL V5 TESTS · 2026-10-14

116 相关后端；另 1 凭据门禁；103 前端

ONE RESPONSE · TWO DIFFERENT GATES

HTTP 200 成立。客户端仍有权拒绝展示。

01 / PERSISTED SYNTHETIC RUN

PostgreSQL

读取同一历史 Top-3 与 Evidence Bundle

→

02 / BACKEND GET API

200

HTTP 响应成功，不代表内容可信

→

03 / CLIENT TRUST VALIDATION

来源闭包

当前命题实际引用了哪些来源?

→

04 / BUSINESS CONTRACT

REJECTED

响应夹带了未引用的候选来源

ROOT CAUSE

服务端来源集合超出展示语义

FIX · NOT A CLIENT BYPASS

全部引用仍校验，只输出实际引用来源

ACCEPTANCE

历史 PG → GET → 可信客户端 → 浏览器

修复后：同一历史响应 3/3 通过客户端契约，浏览器可展示；MatchRun 排名快照前后一致。

HTTP 200 与客户端拒绝曾同时成立。这里是已有 synthetic Fresh Run 的只读回读，不是新的 Qwen 或排序执行。

A RUN LOG IS A DECISION RECORD

工具给出失败类型。工程师决定下一步。

保存的 V5 命令记录把实现未完成、测试夹具违约、环境启动错误与真实数据契约失败分开处理。

01 / PYTEST

4 failed

Service 尚不存在：预期 implementation RED。

02 / VITEST

fixture mismatch

缺少 requirement 字段；补齐夹具，不弱化客户端契约。

03 / TOOL ENV

spawn EPERM

esbuild 子进程未启动；不能把 0 个测试执行算成 FAIL。

04 / REAL DATA

client rejected

历史 PostgreSQL 读回后，来源闭包不成立。

NEXT DECISION / 环境失败重试执行；夹具失败修夹具；真实来源违约修服务端投影，保留客户端拒绝。

NEWLY EXECUTED ENGINEERING DEMONSTRATION

让独立检查器拒绝“看起来完整”的来源。

复用真实 V5 Service 的 synthetic fixture；只对复制的响应注入反例，不改历史数据库。

BEHAVIORAL RED

5 failed

检查器尚未实现

→

GREEN

5 passed

Service payload + 4 controlled mutations

UNCITED EXTRA

多余 descriptor

MISSING CITED

缺失来源

DUPLICATE

重复来源

FOREIGN RANK

跨候选引用

NEW CHECKER / audit.py

```
if used - delivered: raise SourceClosureError("missing_descriptor")
if delivered - used: raise SourceClosureError("uncited_descriptor")
```

PHASE 11 / REVIEWING AN AI-ASSISTED EVALUATION REPORT

我审查的不只有数字，还有数字允许你说什么。

Dev 先选策略；Test 只观察。把 Test 里的观察顺序叫“leader”，读者可能误以为最终 Test 又选了一次模型。

BEFORE / TEST FIELD

~~leader~~

一个词混淆“预先的选择”和“事后的观察”。

风险：把 Test 当作模型选择器。

AFTER / ACTUAL d98a16f CHANGE

DECISION `predeclared_dev_leader`

OBSERVATION `observed_test_ordering`

```
observed = bounded[cohort].pop("leader")
"reselection_allowed": False
```

B / REPRESENTATIVE CODEX REVIEW REQUEST · RECONSTRUCTED FROM THE REAL FIX

“先查 Test 报告是否把观察结果写成模型选择。保持标签、指标函数与冻结 Test 不动；只修报告语义，再加断言：Dev leader 与 observed Test ordering 必须同时出现，而且不得重选。”

实际修复改了表达，没有改分数。 Git diff 增加 split-aware 字段与审计测试；它证明修复存在，不证明原始逐字 Codex 对话或错误由 Codex 产生。

ACTUAL VERIFICATION TIERS

交付不是 Agent 自评。是独立证据同时成立。

旧 Run 与新夹具分开验收；没有验证的远端和生产结论，仍留在边界之外。

01 / HISTORICAL V5 · SAVED READBACK

Fresh Top-3：PG 3/3 → client 3/3

历史 synthetic Run 只读回读；浏览器检查与候选顺序、LTR 原始分数、指纹不变量有保存证据。

02 / 先前审计 · 2026-10-18

新 synthetic Run：独立 tmpfs PG → GET → trusted client → 浏览器

Alembic；31 项持久化测试 + 1 项 fixture seed。GET 2/2；1440 / 390px 候选切换与来源展开。

非旧 Fresh Top-3 重跑；未调用 Qwen，未运行 Ranking。

03 / STILL NOT VERIFIED

remote clean clone • GitHub 深链 • 生产招聘效果

V5 远端复现未验证。本发布候选默认 Turbopack 构建通过；先前 junction 快照默认构建受阻。

AFTER THE ARCHITECTURE TURN

一次真实 Provider 链路，把边界逐项跑通。

冻结的 synthetic Fresh Top-3 记录了 extraction、semantic selection、Grounding、Proposition、持久化回读与排序不变量。

FROZEN JSONL SUMMARY

3 / 3

Grounded Explanation readback

STRUCTURED RECRUITER OUTPUT

3 / 3

Canonical RecruiterProposition stages PASS

RANKING INVARIANCE

ORDER ✓
RAW SCORE ✓
FINGERPRINT ✓

解释不反向修改排序。

ACTUAL MATCHRUN TRACE / PERSISTED

Frozen input → Hybrid + LTR → MatchRun

候选顺序、LTR raw score 和 fingerprint 保留在历史运行结果中。

EXPLANATION VIEW / DERIVED STAGE METADATA

Evidence → Selection → Binder → Grounding

UI 中的这些阶段标签不冒充持久化 AgentStep；ExplanationRun 有独立持久化回读。

ONE CONTROLLED E2E.

真实 Provider 路径在该 synthetic Top-3 上得到 3/3 解释与 3/3 Proposition；不是生产可靠性、在线效果或零幻觉比例。

STRUCTURE VALID ≠ FACT SUPPORT

模型能写完整句子，不代表它有权确认事实。

HANDLE / SCHEMA

8 / 8

8 个合成案例的结构与 Handle 均有效

CASES WITH UNSUPPORTED ITEM

6 / 8

6 / 8 个案例至少 1 条；共 9 条已确认未获支持

EXPERIMENTAL BRANCH

Free-text Narrative

看似自然，却可能把弱关联写成强匹配，或在问句中预设未经证实的经验。

Requires Human Review

TRUSTED MAIN PRODUCT

Structured Proposition

来源、未知与人工核实目标保持分明；正式 Recruiter View 采用这条主路径。

Human decision support

最终 triage 是小规模 synthetic 检查；不是总体幻觉率。独立 Bad Case Review 截图不是 Fresh Top-3 故障。

THE ENGINEER BEHIND PEOPLEMATCH

从产品判断，到最后一次工程验收。

01 / PRODUCT I BUILT

Recruiter View + 要求工作台

Hybrid、十项特征、LambdaMART、Evidence 与人工核实界面连成可操作产品。

02 / AGENT TASK I LED

Mission → 代码 → 审查 → 浏览器

限定冻结边界；拒绝错误引用；让真实 PostgreSQL 回读决定能否交付。

03 / ARCHITECTURE I CHANGED

模型选语义，程序守事实

Phase 9 收回 identity、rank 与 refs 的生成权；Grounding 继续核来源。

THREE QUESTIONS WORTH ASKING

为什么 HTTP 200 后页面仍拒绝？如何让 Test 不参与选型？为什么不能让 Qwen 重写已经确定的身份和引用？

GITHUB SOURCE IN THIS REVISION

github.com/rM0on/peoplematch-agent ↗

本版本包含 V5 源码与测试；历史运行与生产效果仍分别标注。精选公开证据请见仓库的版本化入口。